A Custom Gated DeltaNet Decode Kernel

Register pressure, state bandwidth, and serving tradeoffs on an H100

Mihal Dimo

github.com/mhdimo/qwen38-h100-lab

1 October 2026

Abstract

We implement a CUDA replacement for Qwen3.8-27B's packed Gated DeltaNet (GDN) single-token recurrence. Four warps divide each state tile, lowering compiler register usage from 205 per thread in the profiled reference to 40 in the selected candidate. Three tile sizes are numerically tested and timed. Large-batch gains are limited by the cost of reading and writing the FP32 recurrent state. Against an otherwise matched, already-tuned vLLM engine, measured decode-heavy throughput changes by -0.04% and mixed throughput by +1.38%. These are incremental kernel results, separate from the earlier concurrency tuning and SiLU–FP8 experiments. Production services remain unchanged.

1 Measured target and pinned environment

The H100 80GB HBM3 has driver 610.57.04 and a 700 W power limit. The runtime is vLLM 0.26.0, PyTorch 2.11.0, Triton 3.6.0 and CUDA 13.2.86. We keep native FP8 weights, BF16 activations and FP32 recurrent state. The cached model revision is `017b9c7af6b5689d5dd426a76e0bc077eb5ca20a`.

A prior warmed reference capture identified GDN packed recurrence as 20.45% of summed GPU kernel time in ten steady decode graphs: 480 calls averaging 66.50 μs. The installed source hash was rechecked before this pass and matched the saved reference. These percentages are sums of GPU durations, not exclusive fractions of wall time. Nsight Compute hardware counters were not collected.

Each of 48 GDN layers uses 48 value heads with a 128×128 FP32 state. Sixteen Q/K heads are shared in groups of three value heads. At batch 32, a single layer reads and writes about 201.3 MB of state per step. Dividing this by the reference profile time gives an approximate 3.03 TB/s state-traffic rate; this is an arithmetic estimate, not a measured DRAM counter. It explains why reducing register use need not yield a large concurrent-throughput improvement.

2 Recurrence and CUDA mapping

For one value head, normalize q, k with epsilon 10^{-6}, apply the query scale, and compute

$$d = \exp[-\exp(A)\ \mathrm{softplus}(a + \Delta)], \quad \beta = \mathrm{FP32}(\mathrm{BF16}(\mathrm{sigmoid}(b))),$$

$$\bar{H} = dH, \qquad u = \beta(v - \bar{H}k), \qquad H' = \bar{H} + uk^T, \qquad o = H'q.$$

The intermediate BF16 rounding of β is preserved explicitly. State updates stay FP32; the output is BF16. The operation was already fused in the reference. Our change is the thread/tile mapping and vectorized state access, not a new fusion of previously separate operators.

Method references: NVIDIA CUDA Best Practices; vLLM CUDA graphs. The pinned numerical reference is the installed vLLM v0.26.0 operator; its upstream source can be obtained from github.com/vllm-project/vllm.

Implementation and numerical checks

The selected CUDA kernel uses four warps per block and four value rows per warp: a 16×128 state tile. Each lane owns four adjacent key dimensions. Aligned `float4` loads/stores move the state, and warp shuffles reduce dot products. There is no shared-memory reduction or block-wide barrier. The compiler reports 40 registers/thread, zero spill loads/stores and zero barriers. The alternative 32- and 64-row tiles use 62 and 104 registers/thread, respectively. The original Triton kernel uses one warp for a 32-row tile and 205 registers/thread.

The process-local worker preloads the extension before warmup and CUDA graph capture. It replaces only packed non-speculative decode inside the existing opaque attention operation. Prefill, full attention, weights and installed packages are unchanged. Unsupported layouts and dtypes use the original operator. Positive state slots must be unique and in range; nonpositive padding slots yield zero output without state modification. Strided input/gate rows, strided indices and padded state-slot storage are supported.

Each of the three tiles passes 42 matrix cases over three seeds, batches 0–128, contiguous and padded layouts, BF16/FP32 gate bias, random and zero initial states. 6 additional cases cover int32 indices, unnormalized Q/K, zero Q/K, extreme gates, unaligned-state fallback and FP16 fallback. We compare both installed vLLM and an independent PyTorch recurrence. Untouched slots must remain exactly unchanged. Predeclared tolerances are FP32 state rtol 3×10^{-4} / atol 3×10^{-5} and BF16 output rtol 0.016 / atol 0.002; they were not relaxed to obtain a pass.

Each tile also passes 256 CUDA-graph recurrent steps with changing inputs and independently updated reference state. Maximum trajectory differences are 8.34e-06 in state and 0.000977 in output. This is tolerance-based equivalence over tested inputs, not bitwise equality or an unrestricted error bound. Compute Sanitizer 2026.1.1 reports zero memory errors across 48 direct custom-kernel calls and eight graph replays, including both index/bias types, normalization modes, padding and an upper-bound guard. Instrumentation is filtered to the custom GDN kernels; it does not certify the entire vLLM runtime.

3 Warmed CUDA-graph microbenchmark

Graphs contain eight operations on independent states. Ten alternating trials time 30 graph replays each, divided by 240 calls. The table reports median average time per call for the selected 16-row tile. Loading, compilation and warmup are excluded. This fixed-input recurrence converges toward steady state; the full engine comparison separately exercises changing activations.

Batch	vLLM (μs)	Custom (μs)	Speedup
1	5.97	4.73	1.263×
2	9.45	7.99	1.183×
8	23.29	21.04	1.107×
16	39.97	37.93	1.054×
32	72.88	71.54	1.019×
64	139.49	138.64	1.006×
128	272.71	272.52	1.001×

Active state pools are 25.2 MB at batch 1, 201.3 MB at batch 8 and 805.3 MB at batch 32. Batch 1 can benefit from L2 residency; large batches exceed L2. Allocated pool sizes include unused sentinel slots and are reported separately in the raw data. CUDA-event measurements include graph scheduling and submission gaps. Tile selection uses the exploratory sweep; raw timings for all variants are retained. Tiny speed differences should not be interpreted as universal wins.

4 Matched full-model experiment

Both configurations use 64 sequence capacity, 16 GiB cache, a 16,384-token budget, chunked prefill and `FULL_AND_PIECEWISE` graphs. Only the GDN worker and its compilation-cache directory differ. The prior custom SiLU kernel is absent from both, so its gains are not combined with these results.

The custom trace contains 480 GDN CUDA calls across 10 steady decode graph launches and zero reference packed-decode calls in that capture. This verifies real engine dispatch. Profiled requests do not enter the throughput table. Its mean GDN duration is 66.74 μs versus 66.50 μs in the earlier reference capture: essentially unchanged at batch 32 despite fewer registers.

Each configuration completes 400 measured requests: two repetitions of 128/128 input/output tokens at concurrency one, 4096/1 at concurrency 32, 128/128 at concurrency 32 and 1024/128 at concurrency 32. Seeded prompt tokens and fixed output lengths match; the harness rejects partial output and prefix-cache hits.

Workload / throughput	Tuned vLLM	GDN CUDA	Change
Single / output tok/s	72.31	72.82	+0.71%
Prefill / input tok/s	18168.43	18136.11	-0.18%
Decode / output tok/s	1642.53	1641.93	-0.04%
Mixed / output tok/s	989.13	1002.76	+1.38%

	TTFT p95 (ms)		TPOT p95 (ms)	
Workload	vLLM	Custom	vLLM	Custom
Single	101.69	98.39	13.15	13.08
Prefill	8159.12	8090.34	—	—
Decode	339.67	335.08	17.02	17.04
Mixed	1909.02	1858.83	18.40	21.48

TTFT includes scheduling and queueing. TPOT is the per-request mean interval after its first token, not an individual-token tail. A one-token response has no decode TPOT. Prefill-only throughput acts as a control because this kernel does not change prefill computation; any difference is run variation. Maximum end-of-case GPU memory samples are 50.97 GiB for the reference and 50.97 GiB for the candidate; these are not allocation peaks. The mixed-workload TPOT p95 regresses from 18.40 to 21.48 ms, despite higher aggregate throughput. Concurrent decode throughput is effectively flat.

Generated-token hashes agree for 232/400 candidate/reference requests. Two unchanged reference runs agree for 231/400. Both implementations pass the three text smoke checks. The reference control demonstrates engine nondeterminism but does not prove all candidate differences harmless. Production text, image/video and structured-output quality remain unvalidated.

5 Interpretation and reproducibility

Lower register pressure produces clear small-batch operation gains, while state bandwidth limits large-batch gains. Two sequential repetitions, synthetic traffic, unlocked GPU clocks and scheduling variation limit confidence in small end-to-end differences. These results do not establish the globally fastest kernel or a production deployment recommendation. AsyncLLM throughput bypasses the existing application admission lock and is not production HTTP capacity. For the stated many-user decode objective, we retain stock GDN as the preferred baseline: this candidate does not demonstrate an end-to-end decode win. The CUDA implementation remains an opt-in research artifact for further experiments.

All CUDA source, failed attempts, raw JSON and compiler logs are retained in this repository; large binary profiler artifacts are kept in a local archive. The temporary GPU workspace was hash-verified and removed after the study. `gdn_build_data.py` regenerates tables and source hashes.

```
python gdn_pass/run.py --kernel-action gdn-validate
python gdn_pass/run.py --kernel-action gdn-bench
python gdn_pass/run.py --configs batch64_t16384 \
  batch64_t16384_gdn --repetitions 2
```